Generating Chest X-Rays with a Fully Connected Variational Autoencoder

Cameron Rosenthal
ECE, UC San Diego

Abstract

I implement a variational autoencoder (VAE) in PyTorch and train it on the 1,341 normal chest radiographs in the training split of the Kaggle chest X-ray pneumonia dataset. The encoder and decoder are fully connected networks, images are 128×128 grayscale, and the latent code has 256 dimensions. I report the hyperparameters, training loss curves, the Fréchet Inception Distance (FID) and Inception Score (IS) from pytorch-ignite, and generated images. The samples show the overall anatomy of a pediatric chest X-ray and vary from sample to sample, but they are blurrier than real images and lose fine texture.

1 Introduction

A VAE pairs a decoder $p_\theta(x \mid z)$ with an encoder $q_\phi(z \mid x)$ and trains both by maximizing the evidence lower bound (ELBO) under the prior $p(z) = \mathcal{N}(0, I)$. Decoding $z \sim \mathcal{N}(0, I)$ then generates new images. I generate normal chest X-rays this way and score them with FID and IS.

2 Method

Data. I use the chest X-ray pneumonia dataset from Kaggle. The model trains only on the 1,341 images in `train/NORMAL`; the 234 images in `test/NORMAL` serve only as an FID reference. Each image is converted to grayscale, resized so its shorter side is 128 pixels, center-cropped to 128×128 and scaled to $[0, 1]$. No augmentation is used.

Architecture. The encoder flattens the image into $128^2 = 16{,}384$ pixels and applies two fully connected layers of width 1,024 with ReLU. Two linear heads output the mean $\mu(x)$ and log variance $\log \sigma^2(x)$ of $q_\phi(z \mid x)$, both in $\mathbb{R}^{256}$. The reparameterization trick, $z = \mu + \sigma \odot \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$, lets gradients flow through sampling. The decoder mirrors the encoder ($256 \to 1{,}024 \to 1{,}024 \to 16{,}384$) and ends in a sigmoid so pixels lie in $[0, 1]$.

Objective. For each image I minimize, with $\hat{x}$ the decoder output and the squared error summed over pixels,

$$\mathcal{L}(x) = \|x - \hat{x}\|_2^2 + \beta \,\mathrm{KL}\big(q_\phi(z \mid x) \,\|\, \mathcal{N}(0, I)\big), \qquad \mathrm{KL} = -\tfrac{1}{2} \textstyle\sum_{j=1}^{256} \big(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\big). \quad (1)$$

With $\beta = 1$, the squared error is the negative log-likelihood of a Gaussian decoder with variance $1/2$ up to a constant, so $\mathcal{L}$ is the negative ELBO up to a constant. Summing rather than averaging over pixels keeps the two terms on comparable scales; averaging would let the KL term dominate and the decoder would ignore z.

Training and evaluation. Table 1 lists the settings. I train with Adam and keep the final weights. FID and IS use pytorch-ignite's default `FID` and `InceptionScore` metrics, which run an ImageNet-pretrained InceptionV3 on images upsampled to 299×299 with the gray channel copied into three

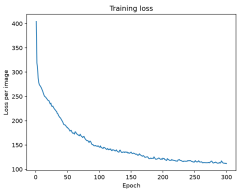
Table 1: Hyperparameter settings.

Setting	Value
Image size	128×128, grayscale
Training images	1,341 (`train/NORMAL`)
Latent dimension	256
Hidden layers	$2 \times 1{,}024$ each, ReLU
Parameters	36.5M
Optimizer	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
Learning rate	10^{-3}, constant
Batch size / epochs	64 / 300
KL weight β	1.0

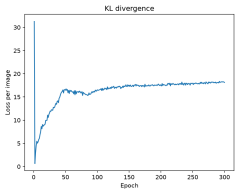
Table 2: FID and IS from pytorch-ignite. FID is measured against `train/NORMAL`.

Images	FID ↓	IS ↑
Real X-rays*	0.024	2.03
VAE samples	0.180	1.74
Untrained VAE	0.203	1.02

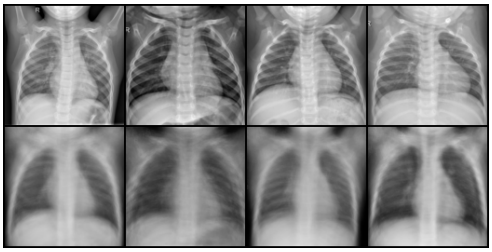
*FID: 234 `test/NORMAL` images against 234 training images. IS: all 1,341 training images.



(a) Total loss



(b) KL divergence



(c) Real (top) and generated (bottom)

Figure 1: (a, b) Training loss and KL divergence per image, averaged over each epoch. (c) Four real training images above four images generated by decoding $z \sim \mathcal{N}(0, I)$.

color channels. I compare 1,341 generated images with the 1,341 training images. Ignite's default FID uses InceptionV3's 1,000 class probabilities, not its 2,048 pooled features, so its values are far below published FIDs. IS rewards confident, varied ImageNet predictions, and X-rays fit no ImageNet class, so IS says little about anatomical realism. For scale, I also score real images and an untrained model.

3 Results

Training curves. Figure 1(a, b) shows the loss per image and its KL term. The loss falls quickly from about 405 in the first epoch to about 150 by epoch 100, then slowly to 112 at epoch 300, still decreasing slightly. The KL term drops to 0.7 in the second epoch, when the decoder briefly ignores z, then rises to about 16 by epoch 50 and settles near 18. It does not collapse to zero, so the decoder uses the latent code. The final squared error of about 94 per image is a root-mean-square pixel error of 0.076 (19 of 255 gray levels).

FID and IS. In Table 2, lower FID and higher IS are better. Real test images score an FID of 0.024 against the training images, roughly the best a model can reach, while an untrained model that outputs noise scores 0.203. The VAE's 0.180 closes only 13% of that gap, consistent with its blur. Its IS of 1.74 is near the real images' 2.03, but as X-rays fit no ImageNet class, this shows little.

Generated images. The samples in Figure 1(c) are recognizable frontal chest X-rays, with two dark lung fields, the spine, curved ribs and the diaphragm, and they differ in brightness, contrast and body size. Next to the real images they are clearly blurrier: rib edges are soft and the fine lung texture is missing.

Limitations. A fully connected decoder has no notion that neighboring pixels belong together, so it must learn each edge separately at every position, and a squared-error loss averages over plausible images when z is uncertain; both blur detail. I track only the training loss, so overfitting is not measured. Next steps are convolutional networks, a learned decoder variance as in the σ-VAE, and a validation loss.